Category	No	Test Item	Test Name
D-Cache	1	TC-D-01	D-Cache STORE→LOAD hit
D-Cache	2	TC-D-02	D-Cache cold miss → refill
I-Cache	3	TC-I-01	I-Cache sequential hit
I-Cache	4	TC-I-02	I-Cache cold miss
Integration	5	TC-INT-01	Boot + ALU execution
Integration	6	TC-INT-02	LOAD/STORE correctness
Integration	7	TC-INT-03	D-Cache miss stall
Integration	8	TC-INT-04	RAW hazard via memory
Integration	9	TC-INT-05	I+D concurrent, no deadlock
Prefetcher	10	TC-PF-01 Miss Trigger Detection	Prefetch stride detect
Prefetcher	11	TC-PF-02 History Buffer & MHT Learning	Miss rate reduction — Prefetch ON vs OFF

Prefetcher	12	TC-PF-03 Domino Prediction & Prefetch Request	No coherence violation — correctness preserved

Description	Test flow
Basic write-then-read on same line (addresses align to same set/way)	1. Issue STORE x12 @ addr 2. Issue LOAD x11 @ same addr 3. Check L1 hit and rdata == wdata
First memory access after reset triggers L1 miss and AXI read transaction	1. Sim reset 2. CPU LOAD from address not in L1 3. Monitor AXI AR channel 4. Wait for data return via RDATA
Sequential instruction fetch on cache hit (no stall, delivers 1 instr/cycle)	1. Reset boots from 0x80000000 (hit in instr SRAM after init) 2. Execute 16 sequential instructions 3. Monitor fetch port output
First instruction fetch from address not in I-Cache triggers miss and memory read	1. Branch to unmapped instruction address 2. I-Cache miss condition triggered 3. AXI AR issued for line refill
System boots from reset and executes ALU program (add/sub/logical ops) with correct trace	1. Load boot code into I-SRAM 2. Reset system, enable clk 3. Execute 32 ALU-only instructions 4. Collect commit trace
Memory operations preserve data semantics: STORE writes to L1/memory; LOAD reads correct value	1. Initialize memory with test pattern 2. LOAD from various addresses 3. STORE to L1, then LOAD same addr 4. Verify against commit log
CPU pipeline stalls correctly when D-Cache miss occurs (no instruction advances until refill)	1. Execute LOAD from uncached addr 2. Monitor EX/MEM stage 3. Check PC advance halts until miss resolves
Write-After-Write and RAW (read-after-write) via memory handled correctly (no stale data)	1. STORE to addr X with value A 2. LOAD from addr X expect value A 3. STORE to addr X with value B 4. LOAD from addr X expect value B
I-Cache and D-Cache operate concurrently on shared AXI bus without deadlock or arbitration failure	1. Execute code with mixed LOAD/STORE/branch 2. Both I-fetch and D-access active simultaneously 3. Monitor AXI arbiter for 100k+ cycles
Domino Prefetcher detects sequential stride pattern (64B per access) and issues prefetch ahead of CPU	1. Execute stride_access.S: loop loads from addr, addr+64, addr+128, ... 2. prefetch_en=1 3. Collect prefetch_req signals
Prefetch reduces D-Cache miss count compared to prefetch disabled on same workload	1. Run stride_access.S with prefetch_en=0, count D-Cache misses 2. Run same with prefetch_en=1, count misses 3. Compare counters

Prefetcher does not corrupt data by prefetching stale/dirty lines during write-heavy workloads	1. Execute write-heavy loop: STORE x11 @ addr loop 2. prefetch_en=1 3. After loop, LOAD same addresses 4. Verify final values

Pass Condition	Priority
dcache_rsp_valid=1 in same/next cycle, rsp_rdata matches stored value, miss_o=0	MUST
miss_o pulse=1 then=0 when read issued; AXI transaction visible in waveform (AR valid→ready→R valid)	MUST
fetch_rtrn.valid = 1 every cycle; fetch_rtrn.data contains correct instr; no gap_o stall	MUST
miss_o = 1 for 1+ cycles; AXI AR issued with correct addr; fetch stall until data returns	MUST
trace_hart_0_commit.log shows PC sequence 0x80000000, +4 per cycle; rd register values correct; no exceptions	MUST
commit log rd_wdata matches expected; memory state at end matches simulation; no silent data corruption	MUST
stall_ex=1 while dcache_miss_o active; PC does not advance; no instruction commits to writeback until data ready	MUST
All LOAD operations retrieve most recent written value; commit log shows no data miscompare; D-Cache coherence maintained	MUST
Simulation completes without timeout; no X on AXI signals; no deadlock detected (stall_ex does not exceed 1000 cycles)	MUST
prefetch_req_valid goes high ≥1 iteration before corresponding CPU LOAD; prefetch_req_addr = base + (iter+1)×64; prefetch_grant=1	MUST
miss_count(prefetch_OFF) > miss_count(prefetch_ON); architectural state identical (no side effects); false prefetch rate <20%	MUST

LOAD returns stored values (no stale prefetch data); no spurious AXI AW transactions; commit log has no unexpected exceptions	MUST